Tula: Technical Design Doc

Personal health assistant. Version 1.0, 8 Oct 2026. Owner: Alok Kumar Mishra.

Summary

Tula is a TypeScript monorepo with an Expo app (Android, iOS and web from one codebase), a NestJS API and PostgreSQL. All AI calls go through a provider interface on the server, so the model can be swapped without touching domain code. Nutrition maths is deterministic and unit tested; the language model only parses inputs and writes copy. The app works offline for logging and syncs through an outbox. Everything runs on free tiers with no card: Render hosts the API, Supabase hosts Postgres and file storage, and a keep-alive ping stops either from sleeping while the app is being evaluated.

1. Goals and non-goals

Goals

- Production-style architecture with clear module boundaries and SOLID design
- One codebase for mobile and desktop web, with platform-appropriate UI patterns
- Reliable AI pipeline: validated outputs, retries, fallback provider, caching
- Offline-tolerant meal logging
- Release APK under 30 MB and a fast cold start

Non-goals

- Authentication, multi-device sync for one user, server push notifications
- HIPAA-grade compliance (documented as a production gap)
- Horizontal scaling beyond a single API instance

2. Stack

Layer	Choice	Reason
App	Expo SDK (latest stable), Expo Router, TypeScript strict	Required by brief; file-based routing; runs on Android, iOS and web
Server state	TanStack Query with persisted cache	Caching, retries, optimistic updates, offline reads
Client state	Zustand	Small, no boilerplate, for UI and outbox state
Local storage	react-native-mmkv (AsyncStorage on web)	Fast synchronous storage for cache and outbox
Animation	Reanimated, react-native-svg	Ring, sheets and count-up on the UI thread
Bottom sheet	@gorhom/bottom-sheet (mobile); custom modal (web)	Native-feeling sheets; modal on desktop
Media	expo-camera,	Capture, compress before

Layer	Choice	Reason
	expo-image-picker, expo-image-manipulator, expo-audio, expo-document-picker	upload, voice, PDFs
Notifications	expo-notifications (local only)	Reminders and recaps without a push server
API	NestJS on Fastify adapter	Modules and dependency injection make boundaries explicit
Database	PostgreSQL on Supabase (free plan, Mumbai region) with Prisma	Relational data, migrations, typed client; pooled URL at runtime, direct URL for migrations; Neon is the fallback if free-tier pausing becomes a problem
Validation	Zod, shared between app and API	One schema for request, response and LLM output
Object storage	Supabase Storage (private buckets)	Meal photos and report files; same provider as the database, no card needed
AI	Gemini Flash (primary), Groq Llama vision (fallback)	Free tier, handles image, PDF and audio input
Hosting	Render free web service (Singapore) for API; EAS Build for APK; Expo web export on Vercel	Free with no expiry; deploys from GitHub; keep-alive ping prevents cold starts
Testing	Jest, Supertest, React Native Testing Library	Unit, API and component tests

3. Repository structure

pnpm workspaces monorepo:

```
tula/
  apps/
    mobile/                 Expo app
      app/                  Expo Router routes
        (onboarding)/
        (tabs)/today.tsx, reports.tsx, chat.tsx
        meal/confirm.tsx
      src/
        features/           one folder per domain
          meals/            api.ts, hooks.ts, components/, outbox.ts
          reports/
          today/
          chat/
          onboarding/
```

```
        ui/                  component kit (Button, Card, Sheet, Ring
...)
        theme/               tokens.ts, typography.ts
        lib/                 apiClient.ts, storage.ts, analytics.ts,
deviceId.ts
    api/                     NestJS app
      src/
        modules/
          profile/  goals/  meals/  reports/  insights/  chat/  media/
        ai/                  provider interface, providers, prompts
        nutrition/           pure calculation functions
        common/              filters, interceptors, guards, config
      prisma/schema.prisma
  packages/
    shared/                  Zod schemas, DTO types, enums, constants
  .github/workflows/ci.yml
```

Rule: features never import from each other. Shared code lives in ui, lib or packages/shared.

4. System architecture

Request flow for a photo meal log:

1. App compresses the photo to about 1024 px, 70 percent JPEG (about 150 KB).
2. App requests a presigned upload URL from POST /media/upload-url and uploads directly to Supabase Storage.
3. App calls POST /meals/parse with the media key and meal type. API checks the parse cache by image hash.
4. On a miss, MealParsingService calls the LLM provider, validates output with Zod and matches items to the curated dish table.
5. App shows the confirmation sheet; user edits and calls POST /meals with an idempotency key.
6. API stores the meal, recomputes the daily summary and returns the updated summary for the ring.

Layering inside the API, top to bottom: Controller (HTTP, validation) to Service (business rules) to Repository (Prisma) and Provider ports (LLM, storage). Controllers hold no logic; services never import Prisma or an AI SDK directly.

5. Data model

Table	Key columns	Notes
users	id (uuid), device_id (unique), name, sex, birth_year, height_cm, activity_level, timezone, created_at	One row per device; seeded demo users
goals	id, user_id, type, start_weight_kg, target_weight_kg, kcal_target,	New row when plan changes; one active

Table	Key columns	Notes
	protein_g, carbs_g, fat_g, fiber_g, water_ml, reasoning, active, created_at	
weight_logs	id, user_id, weight_kg, logged_at	For projection and trend
meals	id, user_id, meal_type, eaten_at, source (photo, voice, text, chat, repeat), media_key, status (pending, parsed, confirmed), idempotency_key (unique), totals jsonb, created_at	Pending allows offline and AI failure
meal_items	id, meal_id, name, dish_ref_id, quantity, unit, kcal, protein_g, carbs_g, fat_g, fiber_g, confidence, edited	edited flag feeds accuracy metric
dish_refs	id, name, aliases text[], unit, kcal_per_unit, protein, carbs, fat, fiber	About 150 curated Indian dishes
reports	id, user_id, file_key, file_hash (unique per user), lab_name, report_date, status, error_code, created_at	Hash prevents duplicate upload
biomarkers	id, report_id, user_id, code, raw_name, value, unit, normalized_value, ref_low, ref_high, status	Indexed on (user_id, code) for trends
daily_summaries	user_id, date, kcal, protein_g, carbs_g, fat_g, fiber_g, meals_count, on_target	Primary key (user_id, date); powers streaks
nudges	id, user_id, rule, message, action, shown_at, acted_at, dismissed_at	Measures nudge action rate
chat_messages	id, user_id, role, content, tool_calls jsonb, created_at	Last 20 messages sent as context
ai_cache	key (sha256 of input and prompt version), response jsonb, created_at	Avoids repeat model calls

6. API design

REST, JSON, versioned under /v1. Every request carries an X-Device-Id header; a guard resolves it to a user. Errors use one shape: { code, message, details? }.

Method and path	Purpose
POST /v1/profile	Create or update profile from onboarding
POST /v1/goals/plan	Compute plan from profile and markers; returns

Method and path	Purpose
	targets and reasoning
GET /v1/today?date=	Summary, meals, active nudge, streak, flagged markers in one call
POST /v1/media/upload-url	Signed Supabase Storage upload URL for photo, audio or report
POST /v1/meals/parse	Parse photo, audio or text into draft items (no save)
POST /v1/meals	Save confirmed meal; requires Idempotency-Key header
PATCH /v1/meals/:id, DELETE /v1/meals/:id	Edit or delete
GET /v1/meals/frequent	Repeat-meal suggestions
POST /v1/reports	Register uploaded report; starts extraction
GET /v1/reports, GET /v1/reports/:id	List, detail with markers
PATCH /v1/biomarkers/:id	User correction of a value
POST /v1/nudges/:id/act, /dismiss	Nudge feedback
POST /v1/chat	Send message; streams reply via server-sent events
GET /v1/health	Liveness for hosting

7. AI layer

7.1 Provider abstraction

```
interface LlmProvider {
  readonly name: string;
  generateStructured<T>(req: {
    prompt: PromptTemplate;
    inputs: Array<TextPart | ImagePart | AudioPart | PdfPart>;
    schema: ZodSchema<T>;
    timeoutMs: number;
  }): Promise<T>;
  streamText(req: ChatRequest): AsyncIterable<ChatChunk>;
}
```

GeminiProvider and GroqProvider implement it. ResilientLlmProvider wraps a list of providers and handles retry, fallback and caching. Domain services receive LlmProvider through dependency injection and never know which model runs. This is the clearest example of dependency inversion and open/closed in the codebase.

7.2 Prompts

- Stored as versioned files in ai/prompts/, for example meal-parse.v2.ts, each exporting a template and its output schema.
- The prompt version is part of the cache key, so changing a prompt invalidates old results.
- Meal parse prompt asks for Indian portion units, a confidence per item, and an isFood flag.

- Report prompt asks for raw name, value, unit and printed reference range for each row; normalisation is done in code, not by the model.

7.3 Reliability

- Output validated with Zod; on failure, one retry with the validation error appended to the prompt.
- HTTP 429 and 5xx: exponential backoff (500 ms, 1.5 s), then fallback provider.
- Timeout 15 seconds for meal parse, 45 seconds for reports.
- All failures end in a defined state (pending meal, failed report with error code), never an unhandled error.
- Per-device rate limit of 30 AI calls per hour to protect the free quota.

7.4 What the model does not do

BMR, TDEE, targets, unit conversion, range comparison, streaks and nudge selection are all plain functions. The model parses input and writes human-readable text. This keeps numbers consistent and testable.

8. Core domain logic

8.1 Nutrition module

Pure functions in nutrition/, no framework imports, fully unit tested:

- calculateBmr(profile), calculateTdee(bmr, activity)
- buildTargets(profile, goal, markers) applies calorie floors, loss-rate cap and marker adjustments
- projectGoalDate(current, target, dailyDeficit)
- matchDish(itemName) fuzzy matches against dish_refs aliases using trigram similarity

8.2 Biomarker module

- Marker registry: code, aliases ("HbA1c", "Glycated Haemoglobin", "A1C"), canonical unit, conversion factors, default range by sex.
- normalize(raw) maps name to code and converts the unit; unknown units are flagged for user review.
- classify(value, range) returns low, in range, high or critical.
- Plausibility checks reject values outside physical limits (for example HbA1c above 20).

8.3 Nudge engine

A list of rule objects, each implementing NudgeRule { id; priority; evaluate(ctx): NudgeCandidate | null }. The engine evaluates all rules against a context (time, summary, targets, markers, streak) and picks the highest-priority candidate not shown in the last 3 hours. Adding a rule means adding a class; the engine does not change. The chosen nudge's text is written by the model from a short template, with a fixed fallback string if the model is unavailable.

8.4 Streaks and summaries

daily_summaries is updated in the same transaction as each meal write. Streak is computed from summaries in the user's timezone, allowing one rest day per ISO week.

9. Frontend architecture

- **Feature folders:** each feature owns its API calls, query hooks, components and types. Screens in app/ are thin and compose feature components.
- **Data fetching:** one hook per resource, for example useToday(date), useLogMeal(). Mutations update the Today cache optimistically so the ring moves before the server responds.
- **Theme:** all values from theme/tokens.ts; a lint rule blocks hex colours and raw numbers in style objects outside the theme folder.
- **Responsive:** a useBreakpoint() hook; the Sheet component renders a bottom sheet under 768 px and a modal above it. Tabs become a sidebar on wide screens through a custom tab layout.
- **Platform files:** where behaviour differs, use .native.tsx and .web.tsx (camera, audio recording, file picking).
- **Device identity:** a UUID generated on first launch and stored in SecureStore (localStorage on web), sent as X-Device-Id. A hidden demo switcher on the onboarding screen loads seeded profiles.
- **Error boundary** per route, with a friendly fallback and retry.

9.1 Offline outbox

1. When offline, a confirmed or raw meal is written to an MMKV outbox with its idempotency key and local photo URI.
2. The Today cache is updated optimistically, and the row shows a pending clock icon.
3. A NetInfo listener drains the outbox in order when the connection returns: upload media, parse if needed, save.
4. Failures retry with backoff; after five failures the item shows "Couldn't sync" with a manual retry.
5. Idempotency keys make retries safe.

10. Design patterns and SOLID map

Principle or pattern	Where it appears
Single responsibility	Controllers only map HTTP; MealParsingService only parses; NutritionCalculator only calculates
Open/closed	New nudge rule or new LLM provider added as a new class with no changes to the engine
Liskov substitution	Any LlmProvider (real, fallback, mock) can replace another in tests and runtime
Interface segregation	Separate ports: LlmProvider, ObjectStorage, Clock, MealRepository

Principle or pattern	Where it appears
Dependency inversion	Services depend on ports injected by NestJS, not on Prisma, Gemini or Supabase SDKs
Strategy	Meal parsing by input type: PhotoParser, VoiceParser, TextParser behind one interface
Decorator	ResilientLlmProvider adds retry, fallback and cache around providers
Repository	Prisma access isolated in repository classes
Outbox	Client-side offline queue

11. Edge case matrix

Case	Handling
Camera or microphone permission denied	Explain why, link to settings, offer text input
Photo is not food	isFood false; friendly message; nothing saved
Model returns invalid JSON	One corrective retry, then fallback provider, then pending meal
Rate limit from model	Backoff, fallback, then pending state with background retry
Double tap on Log meal	Button disabled while saving; idempotency key on server
Offline during log	Outbox; optimistic UI; sync later
Meal logged near midnight or in another timezone	eaten_at stored in UTC; day boundaries computed in user timezone
Report upload of a non-report	Classifier step returns not_a_report; error code stored and shown
Same report uploaded twice	SHA-256 hash check per user; link to existing report
Implausible marker value	Held for user confirmation; not used for targets until confirmed
File too large	Client check at 10 MB; server limit as backup
App killed during onboarding	Step state persisted locally; resume
Server unreachable on launch	Persisted query cache shows last known Today with an offline banner
Very long chat message or prompt injection in food text	Input length limit; user text passed as data, never as instructions; output schema enforced

12. Performance and bundle size

- Hermes engine, release build with minification.
- APK split per CPU architecture through EAS (app bundle or per-ABI APKs); target under 30 MB.
- Only two font families and the weights used; icons tree-shaken by importing individual icons.
- No heavy UI kits or charting libraries; ring and sparklines drawn with react-native-svg.
- Images compressed on device before upload; expo-image with caching for thumbnails.
- Lazy-load chat and report screens; Today is the only screen loaded at start.

- API: single GET /today call for the home screen to avoid request waterfalls; indexes on (user_id, eaten_at) and (user_id, code).

13. Security and privacy

- Secrets in environment variables; never in the app bundle. The app talks only to the Tula API, never to the model directly.
- Signed upload and read URLs expire after 5 minutes; storage buckets are private. The Supabase service key lives only on the API.
- Rate limiting per device on all AI endpoints.
- Helmet-equivalent headers and CORS allow-list on the API.
- Free-tier model terms allow provider training on inputs. Acceptable for the assignment; production would use a paid tier with data retention off.
- Production gaps listed in README: authentication, encryption of health data at rest per field, audit logs, data export and deletion.

14. Testing strategy

Level	What	Tooling
Unit	Nutrition calculations, biomarker normalisation, nudge rules, streak logic	Jest
Service	Meal parsing and report extraction with a mock LlmProvider returning fixtures	Jest with Nest testing module
API	Main endpoints against a test database	Supertest, Prisma test schema
Component	Confirm meal sheet, ring, onboarding step	React Native Testing Library
Manual user testing	Scripted day-one and day-three scenarios on a real Android phone and desktop browser	Checklist in README

Test fixtures include ten real Indian meal photos and three sample lab reports (with personal details removed).

15. Observability

- Structured JSON logs with request ID and device ID. GET /v1/health runs SELECT 1 against the database and is pinged every 10 minutes by cron-job.org, which keeps the Render service and the Supabase project awake (Render sleeps after 15 idle minutes; Supabase pauses free projects after about a week without activity).
- Each AI call logs provider, prompt version, latency, token count, cache hit and outcome.
- Client analytics events: onboarding_completed, meal_logged (with source and time to log), item_edited, nudge_acted, report_uploaded.

16. Delivery

16.1 Git workflow

- Conventional commits, small and frequent, for example feat(meals): portion stepper with Indian units, test(nutrition): calorie floor cases.
- One branch per flow, merged into main; tags for each build.
- CI on GitHub Actions: lint, type check and tests for both apps.

16.2 Build plan

When	Work
Thu evening	Monorepo setup, Supabase and Render accounts, first deploy of the health endpoint, shared schemas, Prisma schema and seed, theme tokens, early feedback request
Fri morning	Nutrition module with tests, LLM provider layer, profile and goals endpoints, onboarding screens
Fri afternoon	Media upload, meal parse and save, Log sheet, capture and Confirm sheet
Fri night	Today endpoint and screen, ring, nudge engine, streaks
Sat morning	Report upload, extraction, normalisation, report screens
Sat afternoon	Chat with tools and streaming, local notifications, outbox, web layout
Sat evening	User testing, fixes, APK build, README, Loom recording

16.3 README contents

- Problem, user and product decisions (what was built and what was cut, with reasons)
- Architecture diagram and module overview
- Setup: environment variables (DATABASE_URL pooled, DIRECT_URL, SUPABASE_URL, SUPABASE_SERVICE_KEY, GEMINI_API_KEY, GROQ_API_KEY), database, running the app and API
- AI model choice and fallback, with rate limit notes
- Edge cases handled
- Known limitations and what production would add
- Links: APK, Loom, hosted web build